

Suryagrid AI Phase 1 Implementation Plan

Solar Nowcasting, DSM Penalty Prediction, Fuzzy Risk Classification, and Agentic API Platform

Project Name: Suryagrid AI

Phase: Phase 1 MVP

Status: Software simulation and toy-data implementation

Purpose: Build the base foundation for the entire Suryagrid AI multi-agent solar-energy platform.

Data Mode: Toy/synthetic weather, cloud, irradiance, schedule, and generation data.

Future Compatibility: Designed to later support Solcast, Open-Meteo, NASA POWER, SCADA/inverter telemetry, RL reward engine, consumer discount engine, and real plant integrations.

1. Phase 1 Vision

Suryagrid AI Phase 1 is a solar-energy forecasting and DSM penalty-risk prediction system.

The system uses toy weather, cloud-cover, irradiance, and solar-site data to estimate expected solar generation. It then compares the predicted generation with the scheduled or agreement-based generation value. If the deviation is higher than the allowed DSM threshold, the system classifies it as a penalty risk.

This Phase 1 implementation becomes the base for the full project. It should include the foundation for:

- Agentic architecture
 - Solar generation forecasting
 - DSM penalty classification
 - Fuzzy risk scoring
 - Backend APIs
 - Frontend dashboard
 - API management
 - Security
 - Database design
 - Logging and monitoring
 - Future RL/reward engine
 - Future real data API integration
 - Future plant/SCADA connector integration
-

2. Phase 1 Core Problem

Solar plants often commit or schedule a certain generation value for a time window. However, actual solar generation changes because of:

- Cloud cover
- Solar irradiance variation
- Temperature
- Humidity
- Rain probability
- Wind conditions
- Panel efficiency
- Time of day
- Seasonal changes

If actual or predicted generation deviates from the scheduled value beyond the allowed DSM threshold, the plant may face a DSM penalty.

In Phase 1, this situation will be simulated using toy data.

3. Phase 1 Objective

The objective is to build a working MVP that can:

1. Generate toy weather, cloud, irradiance, and solar-generation data.
 2. Predict solar generation using a deterministic formula-based model.
 3. Optionally train a simple ML model later using the toy dataset.
 4. Compare predicted generation with scheduled generation.
 5. Calculate deviation in MW and percentage.
 6. Apply DSM threshold logic.
 7. Apply fuzzy logic for risk score classification.
 8. Return penalty or no-penalty status.
 9. Estimate penalty cost.
 10. Explain why the risk happened.
 11. Expose results through FastAPI.
 12. Display results on a Next.js dashboard.
 13. Maintain a clean foundation for the full future multi-agent platform.
-

4. Phase 1 Scope

Included in Phase 1

- Toy data generation
- Solar generation prediction

- DSM penalty calculation
- Fuzzy risk classification
- Agentic module structure
- FastAPI backend
- Next.js frontend dashboard
- PostgreSQL database
- TimescaleDB-compatible time-series schema
- Redis cache and rate-limit preparation
- API management layer
- Security foundation
- Configurable DSM rules
- Configurable penalty rates
- Logging
- Basic monitoring
- Docker-based local setup
- Documentation

Not Included in Phase 1

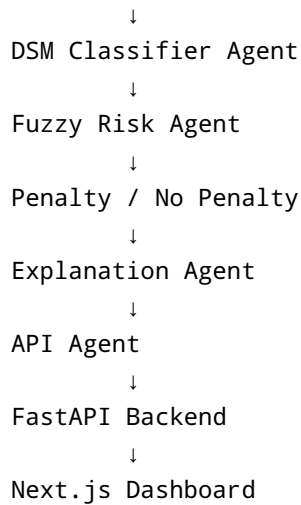
- Real Solcast API integration
- Real Open-Meteo integration
- Real NASA POWER integration
- Real inverter or SCADA integration
- Hardware sensors
- Blockchain
- Full reinforcement learning optimization
- Real-time camera-based cloud nowcasting
- Production payment settlement
- Legal DSM rule enforcement
- Multi-tenant production billing

These will come in later phases.

5. Phase 1 System Flow

```

Toy Weather Data
+ Toy Cloud Data
+ Toy Irradiance Data
+ Solar Site Data
+ Scheduled Generation MW
    ↓
Data Agent
    ↓
Forecast Agent
    ↓
Predicted Solar Generation MW
  
```



6. Recommended Phase 1 Tech Stack

The Phase 1 tech stack should match the main project direction.

Language

- Python 3.12

Reason:

- Best ecosystem for solar-energy modelling, agentic workflows, ML, RL, and backend APIs.

Backend

- FastAPI
- Uvicorn
- Pydantic
- SQLAlchemy
- Alembic
- Pandas
- NumPy
- pvlib
- scikit-learn
- python-dotenv

Reason:

- FastAPI provides typed APIs and automatic OpenAPI documentation.
- Pandas and NumPy handle time-series and toy data.
- pvlib gives a future-ready solar-energy modelling base.
- scikit-learn can be used later for ML forecasting.

Agentic Framework

- LangGraph
- LangChain tools

Reason:

- LangGraph supports explicit agent workflows, state transitions, retries, and quality checks.
- Phase 1 agents can start as simple Python classes and later be connected through LangGraph.

Frontend

- Next.js
- React
- TypeScript
- Tailwind CSS
- Recharts or ECharts
- Axios or TanStack Query

Reason:

- Good dashboard framework for solar timeline charts, DSM status cards, API integration, and future multi-site expansion.

Database

- PostgreSQL
- TimescaleDB extension later

Reason:

- PostgreSQL stores users, sites, schedules, DSM configurations, and audit records.
- TimescaleDB can later optimize solar/weather time-series data.

Cache and API Management

- Redis

Use Redis for:

- API rate-limit counters
- Cache for prediction results
- Agent run state
- Future API key rotation state
- Background job coordination

Security

- JWT authentication for Phase 1
- Later: Auth0, Clerk, or Keycloak

- Secrets stored in `.env` locally
- Future: AWS Secrets Manager or Vault

Deployment

- Docker
- Docker Compose for local development
- GitHub Actions for CI/CD foundation
- Future: AWS ECS/Fargate or Kubernetes

Observability

- Python logging
 - FastAPI middleware logs
 - Structured JSON logs
 - Future: Grafana, Prometheus, LangSmith
-

7. Phase 1 Agentic Architecture

Phase 1 should use a simple agentic architecture. Agents can be implemented as Python classes first. Later, the same agents can be converted into a LangGraph workflow.

7.1 Orchestrator Agent

File:

```
backend/app/agents/orchestrator_agent.py
```

Responsibility:

The Orchestrator Agent controls the full workflow.

It should:

- Start a prediction cycle.
- Ask the Data Agent for toy weather and site data.
- Send data to the Forecast Agent.
- Send forecast result to the DSM Classifier Agent.
- Send DSM result to the Fuzzy Risk Agent.
- Send final result to the Explanation Agent.
- Return the complete result to the API layer.

Input:

- site_id
- timestamp
- scheduled_generation_mw
- DSM configuration

Output:

- complete prediction and DSM report
-

7.2 Toy Data Agent

File:

```
backend/app/agents/toy_data_agent.py
```

Responsibility:

Generate toy weather, cloud, irradiance, and generation data.

It should create data with these columns:

```
timestamp
site_id
irradiance_w_m2
cloud_cover_percent
temperature_c
humidity_percent
wind_speed_mps
rain_probability_percent
scheduled_generation_mw
actual_generation_mw
```

Toy data logic:

- Morning irradiance should be low.
- Noon irradiance should be high.
- Evening irradiance should reduce again.
- Cloud cover should reduce generation.
- High temperature should slightly reduce panel output.
- Random noise should make the data realistic.
- Scheduled generation can be fixed or slightly changing.

7.3 Forecast Agent

File:

```
backend/app/agents/forecast_agent.py
```

Responsibility:

Predict solar generation.

Phase 1 should start with a formula-based model.

Formula:

```
base_generation = solar_capacity_mw * (irradiance_w_m2 / 1000)

cloud_loss_factor = 1 - ((cloud_cover_percent / 100) * 0.75)

temperature_loss_factor = 1 - max(0, temperature_c - 25) * 0.005

predicted_generation_mw = base_generation * cloud_loss_factor *
temperature_loss_factor
```

The final value must be clamped:

```
0 <= predicted_generation_mw <= solar_capacity_mw
```

Later, the Forecast Agent can support:

- Random Forest Regressor
- XGBoost
- pvlib model
- Solcast API forecasts
- Open-Meteo forecasts
- NASA POWER historical data

7.4 DSM Classifier Agent

File:


```
backend/app/agents/dsm_classifier_agent.py
```

Responsibility:

Compare predicted generation with scheduled generation and classify DSM status.

Formula:

```
deviation_mw = abs(predicted_generation_mw - scheduled_generation_mw)

deviation_percent = (deviation_mw / scheduled_generation_mw) * 100

if deviation_percent > allowed_dsm_threshold_percent:
    penalty_status = "PENALTY_RISK"
else:
    penalty_status = "NO_PENALTY"
```

Penalty formula:

```
estimated_penalty_cost = deviation_mw * penalty_rate_per_mw
```

Important:

DSM threshold and penalty rate must be configurable because real DSM rules vary by state, country, DISCOM, and regulator.

7.5 Fuzzy Risk Agent

File:

```
backend/app/agents/fuzzy_risk_agent.py
```

Responsibility:

Give a softer risk score instead of only binary penalty/no penalty.

The fuzzy logic should consider:

- Deviation percentage
- Cloud cover percentage
- Irradiance level

- Confidence score
- Scheduled generation pressure

Risk levels:

```
LOW
MEDIUM
HIGH
CRITICAL
```

Example fuzzy rules:

```
IF deviation_percent < 5
THEN risk = LOW

IF deviation_percent is between 5 and 10
AND cloud_cover_percent is medium
THEN risk = MEDIUM

IF deviation_percent is greater than DSM threshold
AND cloud_cover_percent is high
THEN risk = HIGH

IF deviation_percent is much greater than DSM threshold
AND irradiance is low
AND scheduled_generation_mw is high
THEN risk = CRITICAL
```

Example implementation logic for Phase 1:

```
risk_score = 0

risk_score += deviation_percent * 0.5
risk_score += cloud_cover_percent * 0.2

if irradiance_w_m2 < 300:
    risk_score += 20

if confidence_score < 0.6:
    risk_score += 15

if risk_score < 25:
    fuzzy_risk_level = "LOW"
elif risk_score < 50:
    fuzzy_risk_level = "MEDIUM"
```

```
elif risk_score < 75:
    fuzzy_risk_level = "HIGH"
else:
    fuzzy_risk_level = "CRITICAL"
```

The Fuzzy Risk Agent is important because it becomes the base for the future Reward Agent and RL engine.

7.6 Explanation Agent

File:

```
backend/app/agents/explanation_agent.py
```

Responsibility:

Generate simple human-readable explanations.

Example outputs:

```
"No penalty risk. Predicted generation is close to the scheduled generation and deviation is within the allowed DSM threshold."
```

```
"Penalty risk is high because predicted generation is 1.8 MW lower than scheduled generation. Cloud cover is 72%, which reduced expected solar output."
```

The Explanation Agent should not do math. It should only explain results calculated by deterministic code.

7.7 API Management Agent

File:

```
backend/app/agents/api_management_agent.py
```

Responsibility:

Manage API-related behaviour.

In Phase 1, this agent should handle:

- Request validation
- Response formatting
- Internal API status
- Mock provider selection
- Rate-limit checking using Redis
- Error handling
- API health checking

Future responsibilities:

- Solcast API key rotation
- Open-Meteo fallback
- NASA POWER fallback
- Provider quota tracking
- External API retries
- Failover strategy
- Cost guardrails

7.8 Security Agent

File:

```
backend/app/agents/security_agent.py
```

Responsibility:

Provide security checks for the backend.

Phase 1 responsibilities:

- Validate JWT token
- Validate user role
- Check site-level access
- Prevent invalid payloads
- Block suspicious high-frequency requests
- Log security events

Future responsibilities:

- Multi-tenant isolation
- SCADA connector access policies
- Secrets rotation
- Audit trail verification

- Owner/consumer role management
-

8. Phase 1 Backend Architecture

```
backend/  
  app/  
    main.py  
    config.py  
  
    agents/  
      orchestrator_agent.py  
      toy_data_agent.py  
      forecast_agent.py  
      dsm_classifier_agent.py  
      fuzzy_risk_agent.py  
      explanation_agent.py  
      api_management_agent.py  
      security_agent.py  
  
    api/  
      routes_health.py  
      routes_sites.py  
      routes_toy_data.py  
      routes_prediction.py  
      routes_dsm.py  
      routes_timeline.py  
      routes_auth.py  
  
    core/  
      security.py  
      logging.py  
      exceptions.py  
      rate_limit.py  
  
    db/  
      database.py  
      models.py  
      session.py  
      migrations/  
  
    schemas/  
      site_schema.py  
      toy_data_schema.py  
      prediction_schema.py  
      dsm_schema.py
```

```
auth_schema.py

services/
  site_service.py
  toy_data_service.py
  prediction_service.py
  dsm_service.py
  timeline_service.py
  auth_service.py

utils/
  solar_formula.py
  fuzzy_logic.py
  time_utils.py
  validation.py

data/
  toy_weather_data.csv
  prediction_results.csv

tests/
  test_forecast_agent.py
  test_dsm_classifier.py
  test_fuzzy_logic.py
  test_api.py

Dockerfile
requirements.txt
```

9. Phase 1 Frontend Architecture

```
frontend/
  app/
    page.tsx
  dashboard/
    page.tsx
  sites/
    page.tsx
  predictions/
    page.tsx
  dsm/
    page.tsx
  settings/
    page.tsx
```

```
components/  
  cards/  
    MetricCard.tsx  
    PenaltyStatusCard.tsx  
    FuzzyRiskCard.tsx  
  
  charts/  
    GenerationTimelineChart.tsx  
    DeviationChart.tsx  
    WeatherChart.tsx  
  
  forms/  
    SiteForm.tsx  
    PredictionForm.tsx  
    ScheduleForm.tsx  
  
  layout/  
    Navbar.tsx  
    Sidebar.tsx  
    PageHeader.tsx  
  
lib/  
  api.ts  
  types.ts  
  auth.ts  
  
styles/  
  globals.css  
  
package.json
```

10. Database Schema

Use PostgreSQL in Phase 1. Keep schema compatible with future TimescaleDB.

10.1 sites

Stores solar plant/site details.

```
CREATE TABLE sites (  
  id UUID PRIMARY KEY,  
  name VARCHAR(255) NOT NULL,
```

```
latitude DOUBLE PRECISION,  
longitude DOUBLE PRECISION,  
timezone VARCHAR(100) DEFAULT 'Asia/Kolkata',  
capacity_mw DOUBLE PRECISION NOT NULL,  
panel_efficiency DOUBLE PRECISION DEFAULT 0.18,  
created_at TIMESTAMP DEFAULT NOW()  
);
```

10.2 toy_weather_data

Stores generated toy weather and cloud data.

```
CREATE TABLE toy_weather_data (  
  id UUID PRIMARY KEY,  
  site_id UUID REFERENCES sites(id),  
  timestamp TIMESTAMP NOT NULL,  
  irradiance_w_m2 DOUBLE PRECISION,  
  cloud_cover_percent DOUBLE PRECISION,  
  temperature_c DOUBLE PRECISION,  
  humidity_percent DOUBLE PRECISION,  
  wind_speed_mps DOUBLE PRECISION,  
  rain_probability_percent DOUBLE PRECISION,  
  created_at TIMESTAMP DEFAULT NOW()  
);
```

10.3 generation_data

Stores actual or synthetic generation data.

```
CREATE TABLE generation_data (  
  id UUID PRIMARY KEY,  
  site_id UUID REFERENCES sites(id),  
  timestamp TIMESTAMP NOT NULL,  
  actual_generation_mw DOUBLE PRECISION,  
  predicted_generation_mw DOUBLE PRECISION,  
  created_at TIMESTAMP DEFAULT NOW()  
);
```


10.4 schedule_data

Stores scheduled/agreement generation values.

```
CREATE TABLE schedule_data (  
  id UUID PRIMARY KEY,  
  site_id UUID REFERENCES sites(id),  
  timestamp TIMESTAMP NOT NULL,  
  scheduled_generation_mw DOUBLE PRECISION NOT NULL,  
  allowed_dsm_threshold_percent DOUBLE PRECISION DEFAULT 10,  
  penalty_rate_per_mw DOUBLE PRECISION DEFAULT 15000,  
  created_at TIMESTAMP DEFAULT NOW()  
);
```

10.5 dsm_results

Stores DSM classification output.

```
CREATE TABLE dsm_results (  
  id UUID PRIMARY KEY,  
  site_id UUID REFERENCES sites(id),  
  timestamp TIMESTAMP NOT NULL,  
  predicted_generation_mw DOUBLE PRECISION,  
  scheduled_generation_mw DOUBLE PRECISION,  
  deviation_mw DOUBLE PRECISION,  
  deviation_percent DOUBLE PRECISION,  
  allowed_dsm_threshold_percent DOUBLE PRECISION,  
  penalty_status VARCHAR(50),  
  estimated_penalty_cost DOUBLE PRECISION,  
  fuzzy_risk_score DOUBLE PRECISION,  
  fuzzy_risk_level VARCHAR(50),  
  confidence_score DOUBLE PRECISION,  
  explanation TEXT,  
  created_at TIMESTAMP DEFAULT NOW()  
);
```

10.6 audit_logs

Stores important security and system events.

```
CREATE TABLE audit_logs (  
  id UUID PRIMARY KEY,  
  user_id UUID,  
  action VARCHAR(255),  
  entity_type VARCHAR(100),  
  entity_id UUID,  
  ip_address VARCHAR(100),  
  details JSONB,  
  created_at TIMESTAMP DEFAULT NOW()  
);
```

11. API Design

Base URL:

```
http://localhost:8000/api/v1
```

11.1 Health API

```
GET /health
```

Response:

```
{  
  "status": "ok",  
  "project": "Suryagrid AI",  
  "phase": "Phase 1"  
}
```

11.2 Create Solar Site

```
POST /sites
```

Request:

```
{
  "name": "Toy Solar Plant 1",
  "latitude": 12.9716,
  "longitude": 77.5946,
  "timezone": "Asia/Kolkata",
  "capacity_mw": 10,
  "panel_efficiency": 0.18
}
```

Response:

```
{
  "id": "site_uuid",
  "name": "Toy Solar Plant 1",
  "capacity_mw": 10
}
```

11.3 Generate Toy Data

POST /toy-data/generate

Request:

```
{
  "site_id": "site_uuid",
  "start_date": "2026-06-18",
  "days": 7,
  "interval_minutes": 15
}
```

Response:

```
{
  "message": "Toy data generated successfully",
  "rows_created": 672
}
```

11.4 Get Toy Data

```
GET /toy-data/{site_id}
```

Response:

```
{
  "site_id": "site_uuid",
  "data": [
    {
      "timestamp": "2026-06-18T12:00:00",
      "irradiance_w_m2": 760,
      "cloud_cover_percent": 35,
      "temperature_c": 31,
      "humidity_percent": 62,
      "wind_speed_mps": 3.2,
      "rain_probability_percent": 10
    }
  ]
}
```

11.5 Predict Solar Generation

```
POST /predict
```

Request:

```
{
  "site_id": "site_uuid",
  "solar_capacity_mw": 10,
  "irradiance_w_m2": 720,
  "cloud_cover_percent": 45,
  "temperature_c": 31,
  "humidity_percent": 62,
  "scheduled_generation_mw": 6.5,
  "allowed_dsm_threshold_percent": 10,
  "penalty_rate_per_mw": 15000
}
```

Response:

```
{
  "predicted_generation_mw": 4.65,
  "scheduled_generation_mw": 6.5,
  "deviation_mw": 1.85,
  "deviation_percent": 28.46,
  "allowed_dsm_threshold_percent": 10,
  "penalty_status": "PENALTY_RISK",
  "estimated_penalty_cost": 27750,
  "fuzzy_risk_score": 63.2,
  "fuzzy_risk_level": "HIGH",
  "confidence_score": 0.82,
  "explanation": "Penalty risk is high because predicted generation is lower
than scheduled generation. Cloud cover and temperature reduced expected output."
}
```

11.6 DSM Check

```
POST /dsm/check
```

Request:

```
{
  "predicted_generation_mw": 4.65,
  "scheduled_generation_mw": 6.5,
  "allowed_dsm_threshold_percent": 10,
  "penalty_rate_per_mw": 15000
}
```

Response:

```
{
  "deviation_mw": 1.85,
  "deviation_percent": 28.46,
  "penalty_status": "PENALTY_RISK",
  "estimated_penalty_cost": 27750
}
```

11.7 Timeline API

```
GET /timeline/{site_id}
```

Response:

```
{
  "site_id": "site_uuid",
  "timeline": [
    {
      "timestamp": "2026-06-18T09:00:00",
      "predicted_generation_mw": 3.2,
      "scheduled_generation_mw": 4.0,
      "actual_generation_mw": 3.0,
      "deviation_percent": 20,
      "penalty_status": "PENALTY_RISK",
      "fuzzy_risk_level": "HIGH"
    }
  ]
}
```

12. API Management Plan

The API Management Agent should prepare the system for future real provider integration.

Phase 1 API Management Features

- Request validation
- API versioning with `/api/v1`
- Rate limiting with Redis
- Standard response format
- Standard error format
- Health check endpoint
- Mock provider selection
- API call logs
- Configurable environment variables

Future API Management Features

- Solcast API integration
- Open-Meteo fallback
- NASA POWER fallback
- OpenWeatherMap fallback

- API key rotation
 - Provider quota tracking
 - Provider cost tracking
 - Automatic retry with backoff
 - Failover to backup provider
-

13. Security Plan

Security should be included from Phase 1 because the future project may connect to power plants and sensitive APIs.

13.1 Authentication

Use JWT-based authentication for Phase 1.

Endpoints:

```
POST /auth/register
POST /auth/login
GET /auth/me
```

Roles:

```
admin
operator
viewer
```

13.2 Authorization

Access rules:

- Admin can create sites and manage settings.
- Operator can generate toy data and run predictions.
- Viewer can only view dashboard and reports.

13.3 Input Validation

Use Pydantic schemas to validate:

- capacity_mw must be greater than 0
- cloud_cover_percent must be between 0 and 100
- humidity_percent must be between 0 and 100
- scheduled_generation_mw must not be negative

- allowed_dsm_threshold_percent must be greater than 0
- penalty_rate_per_mw must not be negative

13.4 Secrets Management

For Phase 1 local development:

```
.env
```

Example:

```
DATABASE_URL=postgresql://postgres:postgres@localhost:5432/suryagrid
REDIS_URL=redis://localhost:6379
JWT_SECRET_KEY=change_this_secret
JWT_ALGORITHM=HS256
APP_ENV=development
```

Future production:

- AWS Secrets Manager
- HashiCorp Vault
- No API keys in code
- Automatic secret rotation

13.5 Audit Logging

Log important actions:

- User login
- Site created
- Toy data generated
- Prediction requested
- DSM check performed
- Configuration changed
- Unauthorized access attempt

14. Fuzzy Logic Design

The fuzzy logic layer is used to give a risk level beyond simple penalty/no-penalty.

Inputs

```
deviation_percent  
cloud_cover_percent  
irradiance_w_m2  
confidence_score  
scheduled_generation_mw
```

Output

```
fuzzy_risk_score  
fuzzy_risk_level
```

Risk Levels

```
0 - 24: LOW  
25 - 49: MEDIUM  
50 - 74: HIGH  
75 - 100: CRITICAL
```

Basic Fuzzy Rules

```
Rule 1:  
IF deviation is low AND cloud cover is low  
THEN risk is LOW  
  
Rule 2:  
IF deviation is medium OR cloud cover is medium  
THEN risk is MEDIUM  
  
Rule 3:  
IF deviation is high AND cloud cover is high  
THEN risk is HIGH  
  
Rule 4:  
IF deviation is very high AND irradiance is low  
THEN risk is CRITICAL  
  
Rule 5:  
IF confidence is low  
THEN increase risk level
```

Phase 1 Simple Risk Score Formula

```
risk_score = 0

risk_score += deviation_percent * 0.5
risk_score += cloud_cover_percent * 0.2

if irradiance_w_m2 < 300:
    risk_score += 20

if confidence_score < 0.6:
    risk_score += 15

risk_score = min(100, max(0, risk_score))
```

Risk Level Mapping

```
if risk_score < 25:
    level = "LOW"
elif risk_score < 50:
    level = "MEDIUM"
elif risk_score < 75:
    level = "HIGH"
else:
    level = "CRITICAL"
```

15. Dashboard Design

The frontend dashboard should be simple but future-ready.

15.1 Main Dashboard

Show top cards:

- Solar capacity
- Current predicted generation
- Scheduled generation
- Deviation MW
- Deviation percentage
- Penalty status
- Fuzzy risk level
- Estimated penalty cost

15.2 Timeline Chart

Show lines for:

- Predicted generation
- Scheduled generation
- Actual generation
- DSM threshold band

Use:

- Recharts or ECharts
-

15.3 Weather and Cloud Panel

Show:

- Irradiance
 - Cloud cover
 - Temperature
 - Humidity
 - Wind speed
 - Rain probability
-

15.4 DSM Risk Panel

Show:

- Penalty status
 - Fuzzy risk score
 - Fuzzy risk level
 - Estimated penalty cost
 - Explanation
-

15.5 Settings Page

Allow user to configure:

- Solar capacity
- DSM threshold percentage
- Penalty rate per MW
- Default scheduled generation

- Toy data generation interval
 - Timezone
-

16. Toy Data Generation Design

Toy data should simulate one or more solar days.

Dataset Columns

```
timestamp
site_id
irradiance_w_m2
cloud_cover_percent
temperature_c
humidity_percent
wind_speed_mps
rain_probability_percent
scheduled_generation_mw
actual_generation_mw
```

Generation Logic

Irradiance:

- 0 at night
- low during sunrise
- highest around noon
- low during sunset

Cloud cover:

- random between 0 and 100
- can include cloudy periods

Temperature:

- lower in morning
- higher in afternoon

Actual generation:

```
actual_generation = formula_generation + random_noise
```

Scheduled generation:

```
scheduled_generation can be fixed or based on expected daylight curve
```

17. Solar Prediction Formula

Use this as the first forecasting model.

```
def predict_generation(
    solar_capacity_mw: float,
    irradiance_w_m2: float,
    cloud_cover_percent: float,
    temperature_c: float
) -> float:
    base_generation = solar_capacity_mw * (irradiance_w_m2 / 1000)

    cloud_loss_factor = 1 - ((cloud_cover_percent / 100) * 0.75)

    temperature_loss_factor = 1 - max(0, temperature_c - 25) * 0.005

    predicted = base_generation * cloud_loss_factor * temperature_loss_factor

    predicted = max(0, min(predicted, solar_capacity_mw))

    return round(predicted, 3)
```

This is not a real engineering-grade solar formula. It is a toy approximation for Phase 1.

18. DSM Classifier Logic

```
def classify_dsm_risk(
    predicted_generation_mw: float,
    scheduled_generation_mw: float,
    allowed_dsm_threshold_percent: float,
    penalty_rate_per_mw: float
) -> dict:
    if scheduled_generation_mw <= 0:
        return {
```

```

        "deviation_mw": 0,
        "deviation_percent": 0,
        "penalty_status": "INVALID_SCHEDULE",
        "estimated_penalty_cost": 0
    }

    deviation_mw = abs(predicted_generation_mw - scheduled_generation_mw)

    deviation_percent = (deviation_mw / scheduled_generation_mw) * 100

    penalty_status = (
        "PENALTY_RISK"
        if deviation_percent > allowed_dsm_threshold_percent
        else "NO_PENALTY"
    )

    estimated_penalty_cost = (
        deviation_mw * penalty_rate_per_mw
        if penalty_status == "PENALTY_RISK"
        else 0
    )

    return {
        "deviation_mw": round(deviation_mw, 3),
        "deviation_percent": round(deviation_percent, 2),
        "penalty_status": penalty_status,
        "estimated_penalty_cost": round(estimated_penalty_cost, 2)
    }

```

19. Fuzzy Logic Function

```

def calculate_fuzzy_risk(
    deviation_percent: float,
    cloud_cover_percent: float,
    irradiance_w_m2: float,
    confidence_score: float
) -> dict:
    risk_score = 0

    risk_score += deviation_percent * 0.5
    risk_score += cloud_cover_percent * 0.2

    if irradiance_w_m2 < 300:
        risk_score += 20

```

```

if confidence_score < 0.6:
    risk_score += 15

risk_score = max(0, min(100, risk_score))

if risk_score < 25:
    level = "LOW"
elif risk_score < 50:
    level = "MEDIUM"
elif risk_score < 75:
    level = "HIGH"
else:
    level = "CRITICAL"

return {
    "fuzzy_risk_score": round(risk_score, 2),
    "fuzzy_risk_level": level
}

```

20. Explanation Generator

```

def generate_explanation(
    penalty_status: str,
    predicted_generation_mw: float,
    scheduled_generation_mw: float,
    deviation_percent: float,
    cloud_cover_percent: float,
    irradiance_w_m2: float,
    fuzzy_risk_level: str
) -> str:
    if penalty_status == "NO_PENALTY":
        return (
            f"No penalty risk. Predicted generation is "
            f"{predicted_generation_mw} MW "
            f"against a schedule of {scheduled_generation_mw} MW. "
            f"The deviation is {deviation_percent}% and remains within the "
            f"allowed limit."
        )

    return (
        f"Penalty risk detected. Predicted generation is "
        f"{predicted_generation_mw} MW "
        f"against a scheduled value of {scheduled_generation_mw} MW. "
    )

```

```

        f"The deviation is {deviation_percent}%."
        f"Cloud cover is {cloud_cover_percent}% and irradiance is
{irradiance_w_m2} W/m²."
        f"Overall fuzzy risk level is {fuzzy_risk_level}."
    )

```

21. FastAPI Prediction Endpoint Example

```

from fastapi import APIRouter
from pydantic import BaseModel, Field

router = APIRouter()

class PredictionRequest(BaseModel):
    site_id: str
    solar_capacity_mw: float = Field(gt=0)
    irradiance_w_m2: float = Field(ge=0)
    cloud_cover_percent: float = Field(ge=0, le=100)
    temperature_c: float
    humidity_percent: float = Field(ge=0, le=100)
    scheduled_generation_mw: float = Field(ge=0)
    allowed_dsm_threshold_percent: float = Field(gt=0)
    penalty_rate_per_mw: float = Field(ge=0)

@router.post("/predict")
def predict(request: PredictionRequest):
    predicted = predict_generation(
        solar_capacity_mw=request.solar_capacity_mw,
        irradiance_w_m2=request.irradiance_w_m2,
        cloud_cover_percent=request.cloud_cover_percent,
        temperature_c=request.temperature_c
    )

    dsm = classify_dsm_risk(
        predicted_generation_mw=predicted,
        scheduled_generation_mw=request.scheduled_generation_mw,
        allowed_dsm_threshold_percent=request.allowed_dsm_threshold_percent,
        penalty_rate_per_mw=request.penalty_rate_per_mw
    )

    confidence_score = 0.82

    fuzzy = calculate_fuzzy_risk(
        deviation_percent=dsm["deviation_percent"],

```



```

        cloud_cover_percent=request.cloud_cover_percent,
        irradiance_w_m2=request.irradiance_w_m2,
        confidence_score=confidence_score
    )

    explanation = generate_explanation(
        penalty_status=dsm["penalty_status"],
        predicted_generation_mw=predicted,
        scheduled_generation_mw=request.scheduled_generation_mw,
        deviation_percent=dsm["deviation_percent"],
        cloud_cover_percent=request.cloud_cover_percent,
        irradiance_w_m2=request.irradiance_w_m2,
        fuzzy_risk_level=fuzzy["fuzzy_risk_level"]
    )

    return {
        "site_id": request.site_id,
        "predicted_generation_mw": predicted,
        "scheduled_generation_mw": request.scheduled_generation_mw,
        **dsm,
        **fuzzy,
        "confidence_score": confidence_score,
        "explanation": explanation
    }

```

22. LangGraph Future Workflow

Phase 1 agents can start as normal Python services. Later, connect them through LangGraph.

Future graph:

```

START
  ↓
ToyDataAgent / RealDataAgent
  ↓
ForecastAgent
  ↓
DSMClassifierAgent
  ↓
FuzzyRiskAgent
  ↓
ExplanationAgent
  ↓
APIManagementAgent

```

↓
END

With QA loop:

If data quality fails → return to Data Agent
If prediction is out of bounds → return to Forecast Agent
If schedule is invalid → ask user/operator to update schedule

23. Development Milestones

Milestone 1: Repo Setup

Create:

```
backend/  
frontend/  
docs/  
docker-compose.yml  
README.md
```

Set up:

- Python 3.12 environment
- FastAPI
- Next.js
- PostgreSQL
- Redis
- Docker Compose

Milestone 2: Backend Foundation

Build:

- FastAPI app
 - API versioning
 - Health endpoint
 - Pydantic schemas
 - Error handling
 - Logging middleware
-

Milestone 3: Database Foundation

Build:

- SQLAlchemy setup
 - PostgreSQL models
 - Alembic migrations
 - Sites table
 - Toy data table
 - Schedule data table
 - DSM results table
 - Audit logs table
-

Milestone 4: Toy Data Agent

Build:

- Generate synthetic weather data
 - Generate cloud cover
 - Generate irradiance curve
 - Generate actual generation
 - Store data in database
 - Return data through API
-

Milestone 5: Forecast Agent

Build:

- Formula-based solar prediction
 - Clamp prediction to plant capacity
 - Add confidence score
 - Unit tests
-

Milestone 6: DSM Classifier Agent

Build:

- Deviation MW
- Deviation percentage
- Penalty/no-penalty status
- Estimated penalty cost
- Configurable threshold
- Configurable penalty rate

Milestone 7: Fuzzy Risk Agent

Build:

- Risk score
- Risk level
- Risk rules
- Integration with DSM output

Milestone 8: Explanation Agent

Build:

- Simple explanation generator
- Penalty explanation
- No-penalty explanation
- Cloud/irradiance explanation

Milestone 9: API Management and Security

Build:

- JWT authentication
- Role-based access
- Redis rate limiting
- Request validation
- Audit logs
- Standard API errors

Milestone 10: Frontend Dashboard

Build:

- Main dashboard
 - Prediction form
 - Site form
 - DSM result card
 - Fuzzy risk card
 - Timeline chart
 - Weather panel
-

Milestone 11: Testing

Add tests for:

- Forecast formula
- DSM classifier
- Fuzzy logic
- API validation
- Invalid schedule
- Prediction clamping
- High cloud scenario
- Low irradiance scenario

Milestone 12: Documentation

Create:

```
docs/PHASE_1_IMPLEMENTATION.md
docs/API_DESIGN.md
docs/AGENT_ARCHITECTURE.md
docs/SECURITY_PLAN.md
docs/FUZZY_LOGIC.md
```

24. Testing Plan

Test Case 1: Normal Sunny Day

Input:

```
irradiance = 850
cloud_cover = 10
temperature = 30
scheduled_generation = 7
capacity = 10
threshold = 10
```

Expected:

```
High predicted generation
Low deviation
```

```
NO_PENALTY  
LOW or MEDIUM fuzzy risk
```

Test Case 2: Heavy Cloud Cover

Input:

```
irradiance = 400  
cloud_cover = 85  
temperature = 28  
scheduled_generation = 7  
capacity = 10  
threshold = 10
```

Expected:

```
Low predicted generation  
High deviation  
PENALTY_RISK  
HIGH or CRITICAL fuzzy risk
```

Test Case 3: Invalid Schedule

Input:

```
scheduled_generation = 0
```

Expected:

```
No divide-by-zero error  
Return INVALID_SCHEDULE
```

Test Case 4: Prediction Above Capacity

Input:

```
capacity = 10
irradiance = 1200
cloud_cover = 0
temperature = 20
```

Expected:

```
Predicted generation must not exceed 10 MW
```

Test Case 5: Missing Required Value

Input:

```
cloud_cover missing
```

Expected:

```
Pydantic validation error
HTTP 422
```

25. Phase 1 Environment Variables

```
APP_NAME=Suryagrid AI
APP_ENV=development
API_PREFIX=/api/v1

DATABASE_URL=postgresql://postgres:postgres@localhost:5432/suryagrid
REDIS_URL=redis://localhost:6379

JWT_SECRET_KEY=change_this_secret
JWT_ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=60

DEFAULT_DSM_THRESHOLD_PERCENT=10
DEFAULT_PENALTY_RATE_PER_MW=15000
```

26. Docker Compose Plan

Services:

```
backend
frontend
postgres
redis
```

Example:

```
version: "3.9"

services:
  postgres:
    image: postgres:16
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
      POSTGRES_DB: suryagrid
    ports:
      - "5432:5432"

  redis:
    image: redis:7
    ports:
      - "6379:6379"

  backend:
    build: ./backend
    depends_on:
      - postgres
      - redis
    ports:
      - "8000:8000"
    env_file:
      - .env

  frontend:
    build: ./frontend
    depends_on:
      - backend
    ports:
      - "3000:3000"
```

27. README Startup Commands

Backend:

```
cd backend
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
uvicorn app.main:app --reload
```

Frontend:

```
cd frontend
npm install
npm run dev
```

Docker:

```
docker compose up --build
```

Open:

```
Backend: http://localhost:8000/docs
Frontend: http://localhost:3000
```

28. Final Phase 1 Deliverables

By the end of Phase 1, the project should have:

- Working FastAPI backend
- Working Next.js frontend
- Toy data generator
- Formula-based solar prediction
- DSM penalty classifier
- Fuzzy risk classifier
- Explanation generator
- API management foundation
- Security foundation
- PostgreSQL schema

- Redis setup
 - Dashboard charts
 - Docker Compose setup
 - API documentation
 - Agent architecture documentation
 - Test cases
 - README setup guide
-

29. Future Phase Compatibility

Phase 1 should be coded so that later phases can add:

- Solcast API
 - Open-Meteo API
 - NASA POWER API
 - Real plant telemetry
 - SCADA connectors
 - pvlib production-grade modelling
 - RL reward and penalty optimization
 - Consumer discount engine
 - Multi-site support
 - Multi-tenant auth
 - Cloud-camera nowcasting
 - Blockchain energy settlement
 - Hardware edge nodes
 - Production deployment on AWS ECS/Fargate or Kubernetes
-

30. Phase 1 Success Criteria

Phase 1 is successful when:

1. User can create a solar site.
 2. User can generate toy weather and cloud data.
 3. System can predict solar generation.
 4. System can compare prediction with scheduled generation.
 5. System can calculate DSM deviation.
 6. System can classify penalty or no penalty.
 7. System can calculate fuzzy risk level.
 8. System can explain the result.
 9. Backend APIs work through Swagger UI.
 10. Dashboard shows prediction, schedule, deviation, risk, and timeline.
 11. Security and API validation are included from the beginning.
 12. Codebase is ready for real APIs and advanced agents in the next phase.
-

Final Phase 1 Summary

Suryagrid AI Phase 1 is the foundation of the complete solar multi-agent platform.

It should not try to solve everything immediately. It should first prove the core loop:

```
Weather + Cloud + Irradiance + Solar Site
    ↓
Forecast Generation
    ↓
Compare With Scheduled MW
    ↓
DSM Penalty Check
    ↓
Fuzzy Risk Classification
    ↓
Explanation + API + Dashboard
```

Once this works with toy data, the same architecture can be expanded into real weather APIs, Solcast integration, pvlib modelling, RL reward optimization, consumer discounts, and real solar plant integration.